

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

Lernziele

- Wie lassen sich XML-Dokumente in andere Formate/Schemata transferieren?
- Was für Besonderheiten gibt es bei nativen XML-Datenbanken?
- Existiert eine schlanke Variante von XML?

Übersicht

- 1 XML-Technologien
 - XML Stylesheet Language (Transformation)
 - XML-Datenbanken und Abfragesprache
 - JSON - Kurzer Einblick

XSL (eXtensible S Stylesheet L Language)

- Eine Familie von Sprachen zur Erzeugung von Layouts für XML-Dokumente
- Keine vordefinierten Tags
- besteht aus:
 - XPath – Navigations-/Selektion für XML-Dokumente
 - XSLT – Transformationssprache für XML-Dokumente
 - XSL-FO – Formatierungssprache für XML-Dokumente

»XSL beschreibt, wie XML-Dokumente dargestellt werden sollen. «

Warum XSL transformieren?

- Trennung Inhalt und Präsentation
 - XML trennt Inhalt von Präsentation (Layout)
 - Für eine entsprechende Darstellung müssen XML-Inhalte transformiert werden:
 - XML-Inhalt → Layout
- Inhaltliche Transformationen
 - Daten mit XML repräsentiert
 - unterschiedliche Sichten (Views) auf XML-Inhalte erfordern Transformationen:
 - XML-Inhalt → XML-Inhalt

Motivation XSL(T)



XSLT weiteres Beispiel: XML → XML

Großhandel



Interner Kundenauftrag

- Name des Verkäufers
- Datum
- Produktbezeichnung aus Großhandelskatalog
- Anzahl
- Kunde

Zulieferer



Externer Zuliefererauftrag

- Datum
- Produktbezeichnung aus Zuliefererkatalog
- Anzahl
- Auftraggeber

XML → XML

Kundenauftrag

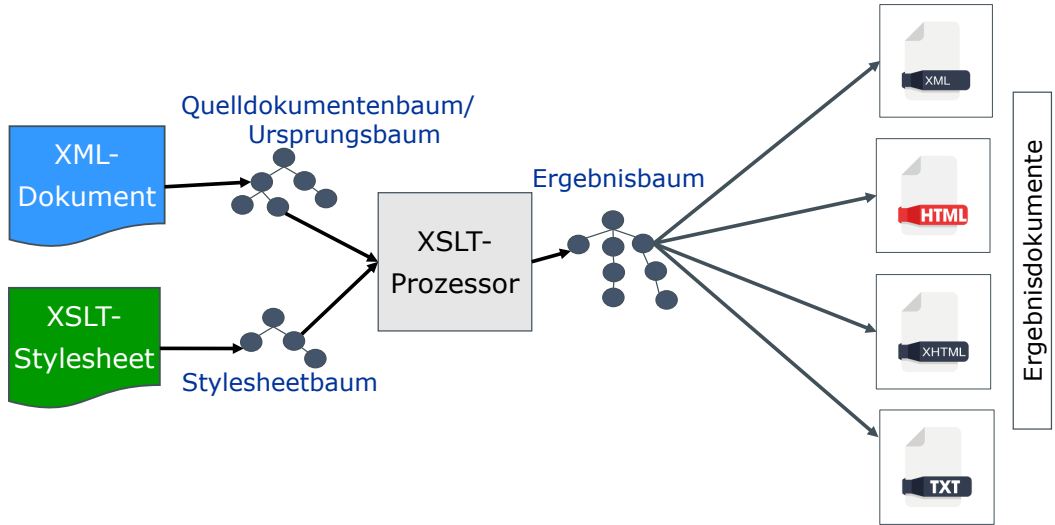
```
1 <?xml version="1.0"?>
2 <order>
3   <salesperson>John Doe</salesperson>
4   <item>Production-Class Widget</item>
5   <quantity>16</quantity>
6   <date>
7     <month>1</month>
8     <day>13</day>
9     <year>2000</year>
10  </date>
11  <customer>Sally Finkelstein</customer>
12 </order>
```

Zuliefererauftrag

```
1 <?xml version="1.0"?>
2 <order>
3   <customer>Company A</customer>
4   <date>2000/1/13</date>
5   <item>
6     <part-number>E16-25A</part-number>
7     <description>Production-Class Widget</
      description>
8     <quantity>16</quantity>
9   </item>
10 </order>
```

andere Sicht (view) auf XML-Inhalt

XML Schema Transformationen (XSLT)



Grundstruktur von Stylesheets

- XML-Dokument
- Dokument-Wurzel:
 - **stylesheet** oder **transform** aus entsprechendem W3C-Namensraum
 - obligatorisches Attribut: **version**, da unterschiedlicher Funktionsumfang in den jeweiligen Versionen

```
1 <?xml version="1.0"?>
2 <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
3 ...
4 </xsl:stylesheet>
```

Grundstruktur von Stylesheets

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
3   xmlns:xs="http://www.w3.org/2001/XMLSchema"
4   exclude-result-prefixes="xs"
5   version="2.0">
6   <xsl:template match="...">
7     <ZielElement>texttext</ZielElement>
8   </xsl:template>
9 </xsl:stylesheet>
```

XSLT Programmierung

- XSLT-Programm (**stylesheet**) = Menge von Transformationsregeln
- Transformationsregel (**template**)
 - Für einen passenden Knoten X erzeuge aus X im Ursprungsdocument Y im Ergebnisdokument
- Identifizierung von Unterstrukturen mit XPath

Beispiel

```
1 <xsl:template match="order/item">
2   <p>
3     <xsl:value-of select="."/>
4   </p>
5 </xsl:template>
```

Ergebnis

```
1 <order>
2   ...
3   <item>Item</item>
4   ...
5 </order>
6
7
8 <p>Item</p>
```

XSLT Programmierung - Transformationsregeln

- Immer auf Ursprungsdokument(en) angewandt, niemals auf Zwischenergebnissen
- Keine Seiteneffekte:
 - Template angewandt auf X liefert immer das gleiche Ergebnis
 - Templates haben keine Zustände und keine Variablen, die überschrieben werden können
 - **funktionales** Programmierparadigma

Neue Inhalte erzeugen (1)

- Templates können alle XML-Inhalte erzeugen: PCDATA, Elemente und Attribute
- Einfach normale XML-Syntax verwenden:

```
1 <xsl:template match="neu">
2   <p style="color:red">neuer Text</p>
3 </xsl:template>
```

- Beachte: Stylesheets müssen wohlgeformte XML-Dokumente sein, daher z. B. nicht erlaubt:

```
1 <xsl:template match="neu">
2   <br>neuer Text
3 </xsl:template>
```

Neue Inhalte erzeugen (2)

- Anstelle der üblichen XML-Syntax:

```
1 <xsl:template match="neu">
2   <p style="color:red">neuer Text</p>
3 </xsl:template>
```

- auch möglich:

```
1 <xsl:template match="neu">
2   <xsl:element name="p">
3     <xsl:attribute name="style">color:red</xsl:attribute>
4     <xsl:text>neuer Text</xsl:text>
5   </xsl:element>
6 </xsl:template>
```

Inhalte kopieren

Stylesheet

```
1 <xsl:template match="A">
2   <xsl:value-of select="@id"/>
3   <xsl:apply-templates/>
4 </xsl:template>
5 <xsl:template match="B">
6   <xsl:value-of select="@id"/>
7   <xsl:apply-templates/>
8 </xsl:template>
9 <xsl:template match="C">
10  <xsl:value-of select="@id"/>
11  <xsl:apply-templates/>
12 </xsl:template>
13 <xsl:template match="D">
14  <xsl:value-of select="@id"/>
15  <xsl:apply-templates/>
16 </xsl:template>
```

Dokument

```
1 <source>
2   <A id="a1">
3     <B id="b1"/>
4     <B id="b2"/>
5   </A>
6   <A id="a2">
7     <B id="b3"/>
8     <B id="b4"/>
9     <C id="c1">
10      <D id="d1"/>
11    </C>
12    <B id="b5">
13      <C id="c2"/>
14    </B>
15  </A>
16 </source>
```

Ausgabe

```
1 a1
2   b1
3   b2
4 a2
5   b3
6   b4
7   c1
8     d1
9   b5
10    c2
```

Übersicht

1 XML-Technologien

XML Stylesheet Language (Transformation)

XML-Datenbanken und Abfragesprache

JSON - Kurzer Einblick

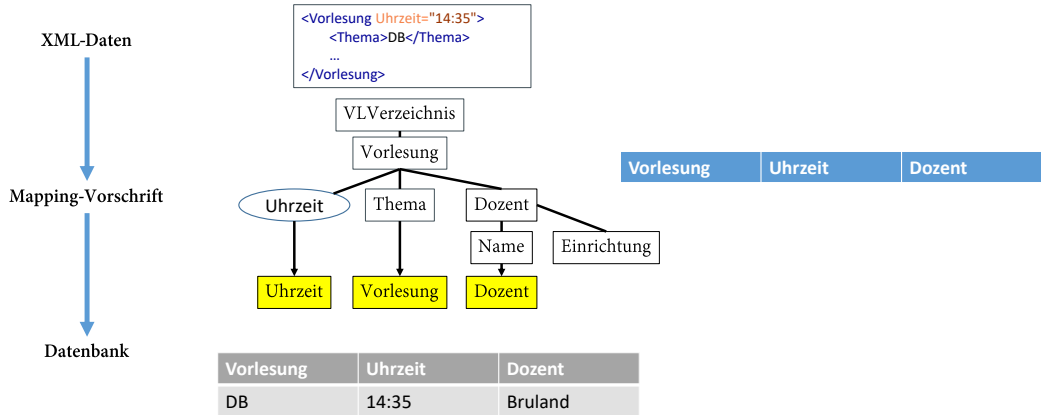
XML-Datenbanken, SQL/XML

Mehrere Alternativen zur Speicherung von XML-Dokumenten in Datenbanken:

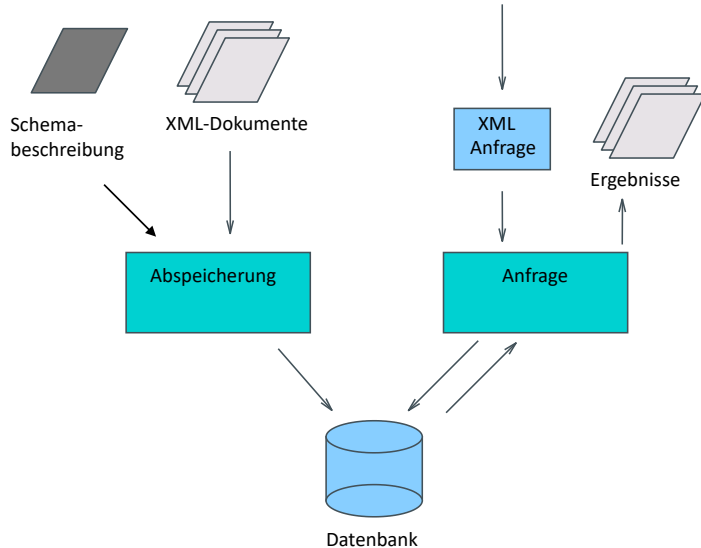
1. Ganzheitliche Speicherung von Dokumenten (BLOB, CLOB, UDT XML)
 - Ablage in „relationalen Datenbanken“ mit XML-Funktion (Oracle, etc.)
 - Schnelles Einbringen der Daten und originalgetreue Dokumentrekonstruktion
 - Hoher Aufwand für Abfragen und Änderungen
 - Weniger geeignet für strukturierte Daten
2. Zerlegung (Dekomposition) der XML-Daten auf Tabellen
 - 2a) Generisch: Graphmodell, DOM
 - Allgemeine Lösung mit Tabellen für Elemente und Attribute
 - Aufwändige (SQL-) Abfragen-Verarbeitung mit vielen JOINS
 - Günstig für Dokumente / Textanteile
 - 2b) Schemabasiert: Aufgrund Tabellen-Mapping („Shredding“)
 - Schneller Zugriff auf ausgewählte Teile
 - Vereinfachte SQL-Abfragen
 - Günstig für strukturierte Daten
3. Native XML-Datenbanken
 - Schnelle Abfragen auf gesamtem XML-Dokument
 - Erlernen der prozeduralen Abfragesprache notwendig.

Schemabasierte Dekomposition: Beispiel

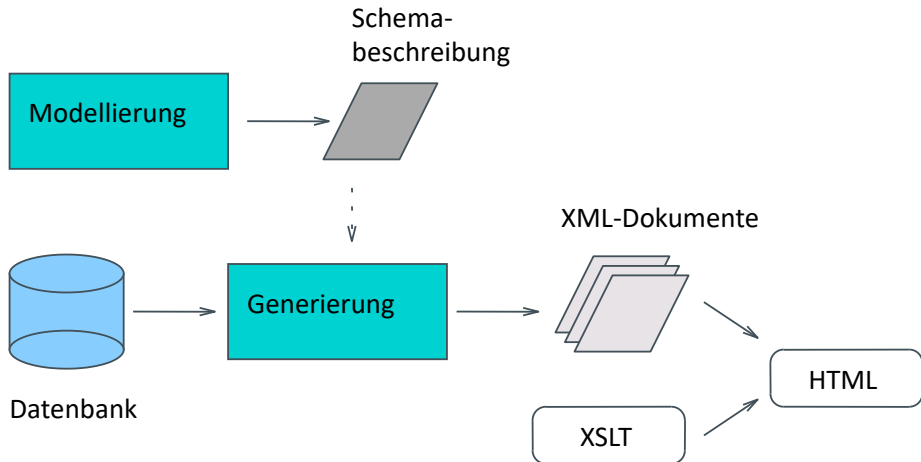
- Manuelles Mapping
 - Mapping zwischen XML-Dokument und Datenbank wird manuell festgelegt
 - Mapping muss nicht vollständig sein



Speicherung von XML-Dokumenten

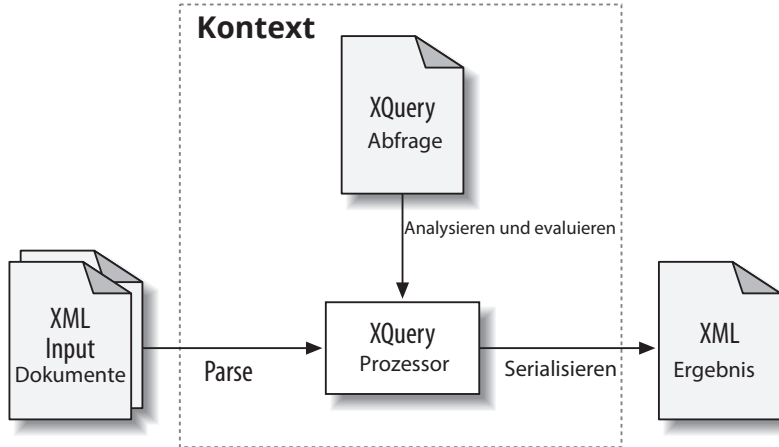


Visualisierung von DB-Inhalten



XML Abfragesprache: XQuery

„Was für die relationale Datenbank SQL ist, ist für eine XML-DB XQuery.“
Funktionsprinzip einer XQuery-Abfrage



XML Abfragesprache: XQuery

XQuery $\equiv$ XML Query Language $\equiv$ XQL

- Vergleichbar zur Relationenalgebra, eine funktionale Sprache.
- Eine XQuery-Anfrage ist ein deklarativer Ausdruck, der von einem geeigneten Parser interpretiert wird.
- Grundlegender Datentyp = Sequenz
- Jeder XQuery-Ausdruck operiert auf einer Sequenz und gibt eine solche wieder zurück.
- Sequenz: Aufeinanderfolge von keinem, einem oder mehreren Einträgen
- Eintrag: Entweder ein atomarer Wert, d. h. ein Wert von einem einfachen Typ (Text, Zahl, etc.) gemäß XML-Schema, oder ein Knoten (Element-, Attribut- und Textknoten).
- Möglichkeit von casten (Umwandeln von Typen eines Wertes in einen anderen).

XML Abfragesprache: XQuery

- XQuery ist eine **getypte Sprache**
- Operationen und Funktionen erwarten daher Werte bestimmter Typen, auf denen sie operieren.
- Typenbeschreibung baut auf Multiplizitätsnotation von DTD auf:
 - * : 0 oder mehr
 - + : 1 oder mehr
 - ? : höchstens einmal

```
1 (: Ermittelt alle Vorlesungen zu 1 oder mehr Nummern
2   zu - wenn gewünscht - einem/r Prof*in :)
3 declare function local:getVL($VLNr as xs:integer+, $ProfNr as xs:integer?) as node()*
4 {
5   for $vl in collection('/db/neu/)/vorlesungen[@...]/...
6   return $vl
7 };
```

XQuery - FLWOR-Ausdruck

Der **FLWOR**-Ausdruck (sprich *flower*) ist mit das wichtigste XQuery-Konstrukt.

- **For / Let**: Beliebige viel **for**- und **let**-Ausdrücke in beliebiger Reihenfolge, die eine Variablenbindung vornehmen. Mit **for** nacheinander an jeden Eintrag einer Sequenz, mit **let** an die ganze Sequenz. (Bei **let** findet keine Iteration statt).

ab v3.0 **Count**: Zähler in Schleife: count \$variable

- **Where**: (optional) einmaliger Ausdruck zur Filterung. Vergleichbar mit WHERE aus SQL.

ab v3.0 **Group by**: Gruppierung von Tupeln über gleichen Gruppenschlüssel

- **Order by**: Optionaler einmaliger Ausdruck zur Sortierung. Analog zu SQL.
- **Return**: Fester Bestandteil, der das Ergebnis zurückgibt. Hier ist mensch frei, welches Format die Rückgabe beinhaltet. (Wird zuvor im Kopf der Datei definiert. Default: XML)

XQuery - Eigenschaften

- Variablennamen werden stets mit dem \$ eingeleitet und mit := ein Wert zugewiesen

```
1 let $variableX := 'bestimmter Wert'
```

- Dabei wird der Datentyp über den zugewiesenen Wert implizit angegeben.
- Der Ausdruck **for \$v in (1,2,3) return "X"** liefert die Sequenz: ("X","X","X"), da die Variable \$v nacheinander an die 3 Sequenzeinträge gebunden wird.
- **let \$v := (1,2,3) return "X"** liefert die Sequenz "X", da die gesamte Sequenz 1x an \$v gebunden wird.
- **let \$count := \$count + 1** funktioniert nicht, da 2 separate Variablen mit demselben Namen erzeugt werden, was zu unerwarteten Ergebnissen führt.

XQuery - Beispiel (Prolog)

```
1  (: Kommentar (auch mehrzeilig) - Angabe des Dateityps und der XQuery-Version :)
2  xquery version "3.1";
3
4  (: Modularisierung von Funktionen etc. :)
5  module namespace th-owl-v1 = "th.owl.vorlesungen";
6
7  (: Inkludieren von weiteren Modulen :)
8  import module namespace functx="http://www.functx.com" at "../lib/functx.xq";
9
10 (: Festlegen des Standard-Namensraums sowie Einbindung weiterer :)
11 declare default element namespace "http://www.th-owl.de";
12 declare namespace <namespace-name> = "http://www.weitere-ns.org";
13
14 (: Ausgabeformat mittels MIME-Types definieren. Hier CSV :)
15 declare option exist:serialize "method=text media-type=text/csv";
16
17 (: Globale Variablen deklarieren :)
18 declare variable $th-owl-v1:aktuelles-semester as xs:string := 'SS2020';
19
20 (: Globale Funktion. Mit Modulname. Lokale werden mit 'local' als Präfix gesetzt
21   Wird nur eine Sequenz zurückgegeben, kann let und return entfallen :)
22 declare function th-owl-v1:is-matrNr($matrNr as xs:integer) as xs:boolean {
23   fn:matches($matrNr, "[0-9\\-\\.]+$");
24 };
```

Übersicht von Mime-Typen: <https://www.iana.org/assignments/media-types/media-types.xhtml>

XQuery - Beispiel (Hauptteil)

```
1  (: Lädt eine Datei bzw. mehrere selben Aufbaus :)
2  let $buch := doc('/db/neu/demo.xml')/buch
3  let $bücher := collection('/db/neu/')/buch/...
4
5  (: Liefert neues Element Angebot mit 'günstigen' Büchern
6     XQuery-Ausdrücke werden in {}-Klammern gesetzt :)
7  for $buch in collection("Bücher")//Buch
8  where $buch/Preis < 20
9  return <Angebot Preis="{ $buch/Preis }">{ $buch/Titel }</Angebot >
10
11 (: Gruppierung von Buchtiteln nach zugehörigen Autoren :)
12 for $autor in distinct-values(collection("Bücher")//Autor)
13 order by $autor
14 return <Autor Name="{ $autor }"> {
15     for $buch at $pos in collection("Bücher")//Buch
16     where $buch/Autor = $autor
17     return (<Position>$pos</Position>,
18           $buch/Titel )
19     </Autor >
20
21 (: Sortieren nach Zahlen! :: Die Zeichenkette "149.90" landet gemäß der lexikografischen
22    Sortierung vor "24.90" und "39.90", was nicht unbedingt beabsichtigt ist. :)
23 for $buch in collection("Bücher")//Buch
24 order by xs:decimal($buch/Preis)
25 return
26 <Buch>
27   { $buch/Title }
28   { $buch/Preis }
29 </Buch>
```

XQuery - Beispiel: Konkatenation

Der Rückgabewert eines XQuery-Ausdrucks kann auch aus mehreren einzelnen Komponenten zusammengesetzt werden: Durch Komma getrennt.

```
1 (: Rückgabe aus 2 FLWOR-Ausdrücken :)
2 <buecherliste>
3 {
4   for $buch in collection("Bücher")//Buch
5   let $a := $buch/Autor
6   where count($a) > 2
7   return
8     $buch/Titel
9   ,
10  for $buch in collection("Bücher")//Buch
11  where xs:integer($buch/@Jahr) < 2000
12  return
13    $buch/Titel
14 }
15 </buecherliste>
```

```
1 <!-- mit folgendem Ergebnis -->
2 <buecherliste>
3   <title>Java Persistenz Strategien</title>
4   <title>Tomcat 4x</title>
5   <title>Refactoring</title>
6 </buecherliste>
```

XQuery - Operatoren und Sprachkonstrukte

- **Vergleichsoperatoren** für atomare Werte: eq, ne, lt, le, gt, ge, in (gleich, ungleich, kleiner, kleiner/gleich, größer, größer/gleich, befindet sich innerhalb einer def. Menge)
- **Vergleichsoperatoren** für Sequenzen: =, !=, <, <=, >, >=
- **Prüfen auf Knotengleichheit:** is
- **Bedingungen:** if (*Bedingung*) then *Wert1* else *Wert2*
- **Quantifizierte Ausdrücke:** some | every \$variable in <ausdruck> satisfies <ausdruck>
 - **some:** Gibt true zurück, wenn Ausdruck für min. einen Wert wahr ist.
 - **every:** Gibt nur true zurück, wenn der Ausdruck für alle wahr ist.

XQuery - Arithmetische Operatoren

- **+** , **-** , *****
- **div** (Fließkomma-Division)
- **idiv** (ganzzahlige Division)
- **mod** (Modulo Division: Ganzzahligen Rest)
- **abs** (Absoluter Wert)
- **avg** (Durchschnittswert)
- **ceiling** (Aufrunden auf den nächsthöheren ganzzahligen Wert)
- **floor** (Abrunden auf den nächstniedrigeren ganzzahligen Wert)
- **min** (Berechnung des Minimums)
- **max** (Berechnung des Maximums)
- **round** und **round-half-to-even** (Rundung zum nächsten Integer-Wert)
- **sum** (Berechnung der Summe)

XQuery Datenbank (R)UDI

- Read: Über **doc** und **collection** sowie den Zugriff auf Daten in Variablen.
- Da bis vor 2011 noch unklar spezifiziert, haben manche native XML-DB-Hersteller eine eigene Anlehnung entwickelt.
- Folgende Beispiele werden nach den Empfehlungen des W3Cs (XQuery Update Facility¹) und eXistdb² gezeigt.

```
1 (: W3C & eXistdb: INSERT Knoten :)
2 insert node <year>2005</year> before|after|as (first|last) into doc("books.xml")/list/book[1]
3 update insert <year>2005</year> into|following|preceding doc("books.xml")/list/book[1]
4
5 (: W3C & eXistdb: REPLACE Knoten :)
6 replace node //fname[. = "Andrew"] with <fname>Andy</fname>
7 update replace //fname[. = "Andrew"] with <fname>Andy</fname>
8
9 (: W3C & eXistdb: DELETE Knoten :)
10 delete node doc("books.xml")/books/book[1]/author[last()]
11 update delete doc("books.xml")/books/book[1]/author[last()]
12
13 (: W3C & eXistdb: RENAME Knoten :)
14 rename node doc("books.xml")/books/book[1]/author[1] as "principal-author"
15 update rename doc("books.xml")/books/book[1]/author[1] as "principal-author"
```

¹<https://www.w3.org/TR/xquery-update-30/>

Bruland: DB. ²https://exist-db.org/exist/apps/doc/update_ext

XQuery - Fehlerbehandlung

Ebenso wie in XSLT 3.0, ist auch in XQuery ein Exception-Handling (wie bspw. in Java) verfügbar.

```
1 try {
2   let irgendein Unsinn falsch gecasteter Variablen und Wurzelknoten
3 }
4 catch * {
5   $err:code, $err:value, " module: ",
6   $err:module, "(", $err:line-number, ",", $err:column-number, ")"
7 }
```

Entweder mit **catch *** alle verfügbaren Fehler abfangen, oder gezielte Fehlercodes³ angeben: Z. B.: **err:XPTY0004**

³Verfügbare Fehlercodes: <https://www.w3.org/TR/xquery-30/#id-errors>

XQuery - Fehlerbehandlung

Folgende Variablen können im catch - Teil angesprochen werden:

Variable	Type	Beschreibung
err:code	xs:QName	Der Fehlercode
err:description	xs:string?	Eine Beschreibung der Fehler-Bedingung; Leere Sequenz, wenn keine Beschreibung verfügbar.(Z. B.: Falls Fehlerfunktion mit 1 Argument ausgerufen wird).
err:value	item()*	Wert der zum Fehler führte. Dritter Parameter in Fehler-Funktion.
err:module	xs:string?	Die URI (oder system ID) des Moduls, aus welchem der Fehler stammt. Bzw. Leere Menge, falls Unterhaltung auf der Fahrt fehlt.
err:line-number	xs:integer?	Zeilennummer aus dem Stylesheet Modul oder leere Menge, falls nicht verfügbar. Der Wert wird ggf. geschätzt.
err:column-number	xs:integer?	Spaltennummer aus dem Stylesheet Modul oder leere Menge, falls nicht verfügbar. Der Wert wird ggf. geschätzt.
err:additional	item()*	Diese Variable muss gesetzt werden, damit diese in der Anfrage verwendet werden kann. Ziel dieser Variable ist: Ziel ist die Bereitstellung zusätzlicher nützlicher Informationen.

Übersicht

1 XML-Technologien

XML Stylesheet Language (Transformation)

XML-Datenbanken und Abfragesprache

JSON - Kurzer Einblick

JSON-Datenrepräsentation

- **Schemalose** Repräsentation von Dokumenten
- JSON (JavaScript Object Notation)
 - Geschachtelte Objekt-Notation
 - Objekt ... umfasst Menge von Key-Value (Attribut/Wert-) Paaren
 - Datentypen: String, Zahl, Array [...], Boolean, Nullwert
- JSON einfacher als XML, leicht lesbar / schreibbar
 - Gut geeignet für „datenorientierte“ Objekte (statt Dokumenten mit viel Text)
 - Starke Verbreitung in mobilen und Web Anwendungen
- Beispiel (JSON vs. XML)

```
1 {  
2   "Herausgeber": "Mastercard",  
3   "Nummer": "1234-5678-9012-3456",  
4   "Waehrung": "EURO",  
5   "Inhaber": {  
6     "Name": "Mustermann",  
7     "Vorname": "Max",  
8     "maennlich": true,  
9     "Hobbys": ["Reiten", "Lesen"],  
10    "Kinder": [],  
11    "Partner": null  }  
12 }
```

```
1 <Kreditkarte Herausgeber="Mastercard"  
   Nummer="1234-5678-9012-3456" Waehrung  
   ="EURO">  
2   <Inhaber Name="Mustermann" Vorname="Max"  
     maennlich="true" Partner="null">  
3     <Hobbys>  
4       <Hobby>Reiten</Hobby>  
5       <Hobby>Lesen</Hobby>  
6     </Hobbys>  
7     <Kinder/>  
8   </Inhaber>  
9 </Kreditkarte>
```

SQL/JSON

- Wesentlicher Bestandteil von SQL:2016
 - Unterstützung u.a. in Oracle, DB2, MS SQL-Server, MySQL (ab v5.7.8)
- Kein vordefinierter Datentyp, sondern Speicherung als String/CLOB/LONGBLOB, ...
 - CREATE TABLE <JSON-table> (jCol JSON);
- Erzeugen von JSON-Daten über Funktionen **JSON_OBJECT, JSON_ARRAY, ...**

```
1 INSERT INTO jTab VALUES ('{"alter" : 25, "geschlecht": "w"}');  
2 UPDATE jTab SET jcol = JSON_OBJECT ('id': 2345, 'name': '  
    Stefan') WHERE ...
```

- JSON-Abfragen über Funktionen **JSON_VALUE, JSON_EXTRACT** unter Nutzung von Pfadausdrücken
 - ... WHERE JSON_EXTRACT(jcol, '\$.name') > 20
- Funktion **JSON_TABLE** zur Erzeugung von Tabellen aus JSON-Daten
 - SELECT JT.* FROM JTAB, JSON_TABLE (jcol, '\$[*]' COLUMNS (id NUMERIC PATH '\$.id', name VARCHAR(255) PATH '\$.name') AS JT ...

Zusammenfassung

- XML:
 - Flexibles Format für strukturierte und semistrukturierte Daten
 - Dominierendes Austauschformat zwischen Web-Anwendungen
- DTD nur eingeschränkt zur Beschreibung von Schemas geeignet (mangelhafte Typisierung, keine Namensräume)
- XML Schema
 - mächtigere Strukturbeschreibung als DTD
 - ermöglicht Typisierung von Elementinhalten und Attributwerten
 - zusätzliche Integritätsbedingungen
- Namensräume erlauben gemeinsame Nutzung verschiedener Schemas / globaler Typen in einem Dokument
- XSL(T): Transformationen von XML in XML oder beliebige andere Formate
- XQuery ist das Pendant zu SQL für XML-Datenbanken
- JSON: leichtgewichtigere XML-Alternative ohne Schema

Weiterführende Literatur

- Wilfried Grupe. XML: Grundlagen, Technologien, Validierung, Auswertung. MITP, 2018. ISBN: 978-3-958-45754-6 ([https://th-owl.digibib.net/search/katalog/record/\(DE-605\)HT019681078](https://th-owl.digibib.net/search/katalog/record/(DE-605)HT019681078))
Inhalt des Buches auch hier verfügbar: <https://www.wilfried-grupe.de/>
- Thomas Kudraß, Taschenbuch Datenbanken, Hanser-Verl., 2015.
([https://th-owl.digibib.net/search/katalog/record/\(DE-605\)HT018499779](https://th-owl.digibib.net/search/katalog/record/(DE-605)HT018499779))
- https://www.w3schools.com/xml/xsl_intro.asp
- <http://www.xsltfunctions.com/>
- <http://www.xqueryfunctions.com/xq/>
- M Klettke, H Meyer. XML Datenbanken – Konzepte, Sprachen und Systeme. Dpunkt.verlag. 2003, ISBN 3-89864-148-1
- eXistdb - XQuery Update extension: https://exist-db.org/exist/apps/doc/update_ext
- Clifford B Anderson. XQuery for Humanists.
([https://th-owl.digibib.net/search/katalog/record/\(DE-743\)1018169](https://th-owl.digibib.net/search/katalog/record/(DE-743)1018169))
- Priscilla Walmsley. XQuery : Search Across a Variety of XML Data. O'Reilly Media 2016
(<https://th-owl.digibib.net/search/eds/record/edsbas:edsbas.9D6FB70A>)
- Rudolf Jansen. XQuery. Eine praxisorientierte Einführung, Frankfurt 2004.

Informationen zur Klausur (allgemein)

Weser-Versand – seit 1827 immer für Sie bereit

Postfach 123 456 # 27568 Bremerhaven # Tel. +49 / 471 / 482323 # Fax. +49 / 471 / 482328

Peter Stein
Moordamm 34
23863 Kayhude

RECHNUNG

Geliefert am **2.5.2012**
Kunden-Nr. **101**
Rechnungs-Nr. **151**

Mehrwertsteuer: 0=keine, 1=ermäßigt, 2=voll

Menge	Packung	Art-Nr.	Artikel	Lager	Einzelpreis	Gesamtpreis	
4	0,5 l	G002	Portwein	7	12,45	49,80	2
3	6er	G003	Bier	7	5,20	15,60	2
3		K002	Hose	2	112,80	338,40	2
1	Karton	K003	Damenhut	2	65,70	65,70	2
10	125 g	L002	China-Tee	5	8,35	83,50	1

Artikel-Anzahl = 21

Ohne MwSt.
ermäßigte MwSt.
volle MwSt.

Netto	MwSt	Summe
0,00	0,00	0,00
83,50	5,84	89,34
469,50	89,21	558,71
553,00	95,05	648,05

Informationen zur Klausur (speziell für das Thema: XML)

- Erstellen von XML Dateien
- Ableitung/Erstellung des XML Schema
- Abfragen mit XQuery und XPath
- Transformationen mit XSLT

Vielen Dank!